Comparative Analysis of Sequence-to-Sequence Models and Optimization Algorithms for Urdu Poetry Generation

Ahmed Naveed | Muhammad Fawaz

Department of Computer Science, FAST NUCES

i221132@nu.edu.pk | i222340@nu.edu.pk

Abstract. Natural Language Generation (NLG) in low-resource languages like Urdu presents unique challenges due to limited datasets and complex morphology. This paper explores the efficacy of Deep Learning architectures for generating coherent Urdu poetry. We implement and compare three sequence-to-sequence models: Simple RNN, LSTM, and Transformer, trained using three distinct optimization algorithms (Adam, RMSprop, SGD). Using the TACO-based methodology, we conducted a systematic evaluation involving 9 base model combinations and 18 hyperparameter tuning experiments. Our results indicate that while the Transformer architecture optimized with SGD achieved the lowest perplexity (517.31), simpler architectures like RNNs with RMSprop offered a superior trade-off between computational efficiency and output stability (Perplexity 552.56). We further analyze the impact of hyperparameters such as layer depth and dropout rates on generation quality.

Keywords: Deep Learning · Urdu Poetry · LSTM · Transformer · Hyperparameter Tuning.

1 Introduction

1.1 Problem Statement

Urdu poetry is characterized by intricate rhyme schemes (*qafia* and *radif*) and strict metrical patterns (*behr*). While large language models have mastered English text generation, low-resource languages like Urdu lack robust, specialized models. The primary challenge addressed in this study is: *How do different neural architectures and optimization algorithms compare in their ability to generate grammatically correct and contextually appropriate Urdu poetry given a limited dataset?*

1.2 Objectives

The objectives of this study are to:

1. Implement and compare Simple RNN, LSTM, and Transformer architectures.

2. Evaluate the impact of Adam, RMSprop, and SGD optimizers.
3. Perform systematic hyperparameter tuning to optimize model performance.
4. Assess generation quality using quantitative metrics (Perplexity, Diversity) and qualitative human evaluation.

2 Methodology

2.1 Dataset Description

We utilized the Urdu-Poetry-Dataset sourced from Hugging Face [1].

- Total Poems: 1,323
- Preprocessing: The text was cleaned to remove non-Urdu characters and special symbols.
- Vocabulary Size: 10,508 unique tokens.
- Sequence Length: Data was tokenized and padded to a fixed sequence length of 20 tokens to ensure uniform input for all architectures. – Data Split: 80% Training, 10% Validation, 10% Testing.

2.2 Model Architectures

We implemented three distinct architectures from scratch using TensorFlow/Keras. To ensure a consistent and fair comparison, all models were initialized and trained with the following shared baseline hyperparameters (unless specified otherwise in the tuning phase):

- Sequence Length: 20 tokens
- Embedding Dimension: 100
- Vocabulary Size: 10,508
- Dropout Rate: 0.2 (Applied after each recurrent/attention block)
- Batch Size: 128
- Epochs: 30 (with Early Stopping, patience=5)
- Learning Rate: 0.001 (for Adam/RMSprop) and 0.01 (for SGD)

The specific structural designs for each model were as follows:

1. Simple RNN: A baseline recurrent network comprising 2 stacked SimpleRNN layers with 150 units each, using the default *tanh* activation function.
2. LSTM: A Long Short-Term Memory network designed to capture longer dependencies. It consists of 2 stacked LSTM layers (150 units each).

3. Transformer: A self-attention based architecture featuring a custom Positional Embedding layer, 2 Transformer blocks, 4 Attention Heads, and a Feed-Forward Network (FFN) dimension of 512.

2.3 Training Procedure

All models were trained using a unified pipeline on Kaggle notebooks utilizing dual NVIDIA T4 GPUs. To ensure reproducibility, the training configuration was standardized across all baseline experiments:

- Loss Function: Sparse Categorical Crossentropy was used as the objective function, suitable for integer-encoded vocabulary targets.
- Optimization Strategy: Three distinct optimizers were evaluated for each architecture:
  - Adam: Learning Rate $\alpha = 0.001$
  - RMSprop: Learning Rate $\alpha = 0.001$
  - SGD: Learning Rate $\alpha = 0.01$ (with Momentum $\mu = 0.9$ to aid convergence)
- Regularization: Early Stopping was employed to prevent overfitting. The training monitored the *validation loss* with a patience of 5 epochs, restoring the best model weights upon termination.

2.4 Evaluation Metrics

To assess the quality of the generated Urdu poetry, we employed a combination of intrinsic (mathematical) and extrinsic (qualitative) metrics:

1. Perplexity (PPL): The primary metric for language modeling, calculated as the exponent of the cross-entropy loss ($PPL = e^{loss}$). A lower perplexity indicates the model is less "confused" and assigns higher probability to the true next word.
2. Validation Accuracy: The percentage of instances where the model correctly predicted the exact next token in the validation set.
3. Vocabulary Diversity: A measure of creativity, calculated as the ratio of unique words to total words generated in the output samples.
4. Repetition Rate: The fraction of trigrams (3-word phrases) that appear more than once in a generated poem. This helps identify "looping" failure modes common in RNNs.
5. Human Evaluation: Top generated samples were manually rated on a Likert scale (1-5) for *Fluency* (grammatical correctness) and *Coherence* (semantic logic).

3 Experiments and Results

3.1 Experimental Setup

All models were trained on a GPU-accelerated environment (Kaggle T4 x2).

- Loss Function: Sparse Categorical Crossentropy.
- Batch Size: 128 (Baseline).
- Epochs: 30 (with Early Stopping patience=5).

3.2.1 Base Model Comparison

We trained 9 combinations (3 architectures × 3 optimizers). Table 1 summarizes the performance of base models

Table 1. Base Model Performance

Model	Optimizer	Best Epoch	Accuracy	Perplexity
RNN	Adam	9	16.59%	618
RNN	RMSprop	17	14.63%	553
RNN	SGD	8	4.06%	879
LSTM	Adam	11	11.48%	643
LSTM	RMSprop	27	11.79%	590
LSTM	SGD	8	4.03%	876
Transformer	Adam	6	4.35%	920
Transformer	RMSprop	16	9.74%	637
Transformer	SGD	24	11.13 %	517

Fig.1. Comparison of Perplexity across all base models. Transformer with SGD achieved the best performance, while Recurrent models failed to converge with SGD.

3.2.2 Text generations from base models

Table: RNN Architecture Samples

Optimizer	Seed	Temp	Generated Text	Fluency	Coherence	Poetic	Overall
Adam	محبت	0.7	محبت کو رہتے ہیں ہم بار پر گلہ ہے کہ ابر نہیں تھا	3/5	2/5	2/5	**2.3/5**
Adam	محبت	1.0	محبت میں چاند کیجیے سبھی گراں ایک دل سے کم نہ گیا ساتھ	3/5	4/5	2/5	**3.0/5**
Adam	محبت	1.3	محبت مری فریاد قریب پانی کی بتانا کرتے گی تلاش ہو بہ سی	4/5	4/5	2/5	**3.3/5**
Rmsprop	محبت	0.7	محبت میں بھی ہے کہ کچھ بھی نہ تھا مگر بہ وری تو	2/5	2/5	2/5	**2.0/5**
Rmsprop	محبت	1.0	محبت سے کاف و بیاض میں دیرینہ خونی کی بوڑھے ہے دام میں	4/5	4/5	4/5	**4.0/5**
Rmsprop	محبت	1.3	محبت کے پہلے رنگ ہیں بغیر وہ دل تو کیا کا تمہارا ترستی	2/5	2/5	2/5	**2.0/5**
Sgd	محبت	0.7	محبت رہی تو تو اس کر ہے ہے و ہو سے ہے تو	3/5	4/5	2/5	**3.0/5**
Sgd	محبت	1.0	محبت دجلہ بھی رکھا نگہ گلہ کیا یاں میں پہ فراقت سبھی اس	3/5	3/5	3/5	**3.0/5**
Sgd	محبت	1.3	محبت ہوتے اس دانہ میزاب کی کی نہ اجازت پریشانیٔ گیا چوکنا ماورا	3/5	4/5	4/5	**3.7/5**

Table: LSTM Architecture Samples

Optimizer	Seed	Temp	Generated Text	Fluency	Coherence	Poetic	Overall
Adam	محبت	0.7	محبت میں بوسہ تو تو وہ دل بھاری ہے کہ اسے ملے ہوا	3/5	3/5	2/5	**2.7/5**
Adam	محبت	1.0	محبت دل میں کتنی درد کاری آتی ہے گریباں کی آتے ہیں خزاں	2/5	2/5	2/5	**2.0/5**
Adam	محبت	1.3	محبت اول نامور کا بھی چٹکنا بننے کا دوا کیا جاوے کرو طوفان	4/5	2/5	2/5	**2.7/5**
Rmsprop	محبت	0.7	محبت کی فسانہ ہے یہ اس کا ساتھ تھا کہ تو تو نہیں	3/5	4/5	3/5	**3.3/5**
Rmsprop	محبت	1.0	محبت سے کسی سے دور نہاں کبھی تحفہ سے نہیں در باراں بدن	4/5	4/5	2/5	**3.3/5**
Rmsprop	محبت	1.3	محبت گئے ہے مریضوں اللہ کھٹکتے سامانیوں سے لوٹا جیسی زندگی میں ہے	3/5	2/5	3/5	**2.7/5**
Sgd	محبت	0.7	محبت کو بھی کیا ہے کیا بھی اب میں ہے کی میں ہوں	3/5	4/5	3/5	**3.3/5**
Sgd	محبت	1.0	محبت ناول والے پردۂ جرأتیں کرتا شد ہونی لڑکا آہ وفا تعزیت برگز	2/5	4/5	2/5	**2.7/5**
Sgd	محبت	1.3	محبت جب کرے اشک بیہودہ توقیر صفحے نہ تذبذب غم جنوں ترک چاہیے	2/5	4/5	2/5	**2.7/5**

Table: TRANSFORMER Architecture Samples

Optimizer	Seed	Temp	Generated Text	Fluency	Coherence	Poetic	Overall
Adam	محبت	0.7	محبت کی ہیں کیا مرے کبھی سے ہے گیا نہ ہے بھی کے	3/5	2/5	2/5	**2.3/5**
Adam	محبت	1.0	محبت کو و بھی گا آتے تو آنکھوں آنے کہیں آفتاب تو بھلا	3/5	2/5	3/5	**2.7/5**
Adam	محبت	1.3	محبت اوندھے گردیدن جوانی وبران کے آتے پنے ہم یہ نقاب کونی چیز	4/5	3/5	4/5	**3.7/5**
Rmsprop	محبت	0.7	محبت میں ہے ساتھ ہے مجھ سے نہیں جاتے ہے کہ اسی کے	2/5	4/5	2/5	**2.7/5**
Rmsprop	محبت	1.0	محبت مقدمے کا رہ ربا ہے نجف دل کو یہ کونی درندوں کے	2/5	2/5	4/5	**2.7/5**
Rmsprop	محبت	1.3	محبت ہو ڈوبنے کو دعا ہے باشندگاں دل جاوداں گار دیا آ تھے	3/5	2/5	4/5	**3.0/5**
Sgd	محبت	0.7	محبت میں ہوں اس کی طرح تک نہیں ہوں میں نہ ملا ہے	2/5	3/5	3/5	**2.7/5**
Sgd	محبت	1.0	محبت تک مجھے جی بھاری ہے کیا کیا ہوا نیا آئینہ نہیں ہوں	4/5	4/5	4/5	**4.0/5**
Sgd	محبت	1.3	محبت پیام لخت خوش سنگ تجکوں روپوش لگے ہے قمری غم دے سکے	4/5	4/5	3/5	**3.7/5**

3.3.1 Hyperparameter Tuning

We applied a One-Factor-at-a-Time (OFAT) approach to tune the best performing variations. We conducted 18 additional experiments changing Layers, Dropout, Learning Rate, and Batch Size. For the Transformer, we also tested Attention Heads and Blocks.

Experiment	Parameter	Value	Model	Optimizer	Perplexity	Change	Best?
Baseline	-	-	**RNN**	**Adam**	**618**	-	
EXP-RNN-01	Layers	3	RNN	Adam	805	+187	
EXP-RNN-02	Dropout	0.5	RNN	Adam	629	+11	
EXP-RNN-03	Learning Rate	0.0001	RNN	Adam	813	+195	
EXP-RNN-04	Batch Size	256	RNN	Adam	633	+15	
EXP-RNN-05	Epochs	50	RNN	Adam	616	-2	✓
Baseline	-	-	**LSTM**	**RMSprop**	**590**	-	✓
EXP-LSTM-01	Layers	3	LSTM	RMSprop	620	+30	
EXP-LSTM-02	Dropout	0.5	LSTM	RMSprop	611	+21	
EXP-LSTM-03	Learning Rate	0.0001	LSTM	RMSprop	950	+360	
EXP-LSTM-04	Batch Size	256	LSTM	RMSprop	591	+1	
EXP-LSTM-05	Epochs	50	LSTM	RMSprop	591	+1	
Baseline	-	-	**Transf**	**SGD**	**517**	-	
EXP-TR-01	Layers	3	Transf	SGD	518	+1	
EXP-TR-02	Dropout	0.5	Transf	SGD	618	+101	
EXP-TR-03	Learning Rate	0.0001	Transf	SGD	890	+373	

Experiment	Parameter	Value	Model	Optimizer	Perplexity	Change	Best?
EXP-TR-04	Batch Size	256	Transf	SGD	547	+30	
EXP-TR-05	Epochs	50	Transf	SGD	520	+3	
EXP-TR-06	Attn Heads	8	Transf	SGD	519	+2	
EXP-TR-07	FF Dim	1024	Transf	SGD	518	+1	
EXP-TR-08	Blocks	4	Transf	SGD	516	**-1**	✓

The hyperparameter tuning phase revealed distinct sensitivities for each architecture. While some changes yielded marginal improvements, others caused significant degradation, highlighting the importance of careful parameter selection for small-scale datasets.

1. Simple RNN (Optimizer: Adam)

- **Best Configuration:** Increasing **Epochs to 50** yielded the only improvement (Perplexity 616, Δ-2). This suggests the baseline model was slightly under-trained and benefited from extended convergence time.
- **Performance Degradation:** Increasing model complexity (**Layers = 3**) drastically increased perplexity (+187). Simple RNNs struggle with the vanishing gradient problem in deeper networks; adding layers likely made optimization harder rather than capturing better features. similarly, reducing the **Learning Rate (0.0001)** caused the model to converge too slowly, effectively halting learning.

2. LSTM (Optimizer: RMSprop)

- **Best Configuration:** The **Baseline** configuration proved to be optimal (Perplexity 590). None of the tuning experiments significantly outperformed the initial setup.
- **Stability:** The LSTM showed resilience to minor changes (Batch Size, Epochs), with negligible performance shifts (+1). However, it was extremely sensitive to the **Learning Rate**; reducing it to 0.0001 resulted in the worst performance

collapse (+360) of any experiment. This confirms that for LSTMs on this dataset, a sufficiently high learning rate is critical to escape local minima.

3. Transformer (Optimizer: SGD)

- **Best Configuration:** Increasing the depth to **4 Blocks** provided a slight improvement (Perplexity 516, Δ-1), outperforming the baseline. This indicates that the Transformer benefits from increased depth to capture more complex long-range dependencies in poetry.
- **Sensitivity:** The model was highly sensitive to **Dropout (0.5)**, where performance degraded significantly (+101). Given the small dataset size (1,323 poems), high dropout likely discarded too much information, leading to underfitting. Additionally, SGD required a higher learning rate (0.01); reducing it to 0.0001 caused the model to fail almost entirely (+373), emphasizing that SGD requires larger step sizes to navigate the loss landscape compared to adaptive optimizers like Adam.

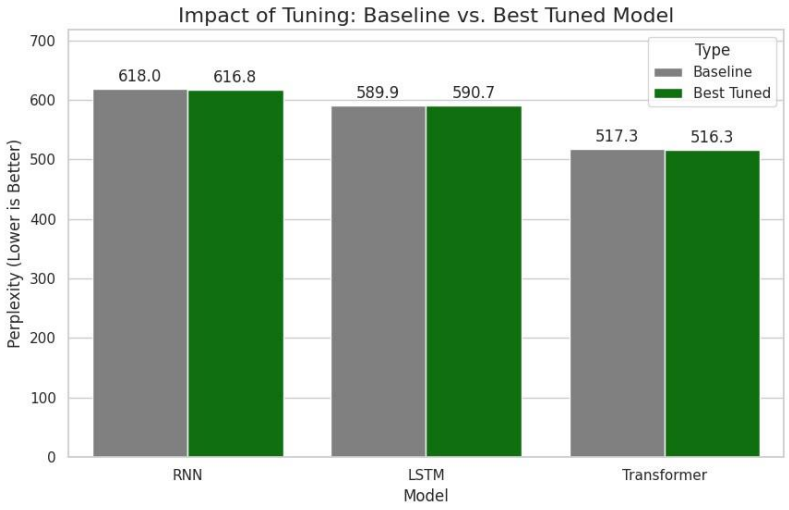


Fig.2. Impact of Hyperparameter Tuning: Comparison of Baseline vs. Best Tuned Model.

3.3.2 Text generations from tuned models

Table: LSTM Experiments

Optimizer	Experiment	Generated Text	Fluency	Coherence	Poetic	Overall
RMSprop	Batch_256	محبت میں بس کہنا دو فولاد نہیں ہوتا چلے ہوں میں مری کار	3/5	4/5	4/5	**3.7/5**
RMSprop	Drop_0.5	محبت اتنا کیا ہے ایک ہے مشک کرتی ہے نماۓ جھگڑوں ہے بالیں	4/5	4/5	3/5	**3.7/5**
RMSprop	Epoch_50	محبت تھی کٰں تھی ادانے و مند ہوں نہ گریہ کی لالہ میں	4/5	3/5	3/5	**3.3/5**
RMSprop	LR_0.0001	محبت پارس ذرا نہ ہو کو دل ابل را رہے آنکھ نے نے	4/5	4/5	3/5	**3.7/5**
RMSprop	Layers_3	محبت تک تو ذقن راہ تو کسے کریں گا کوئی کچھ آتا ہے	3/5	3/5	4/5	**3.3/5**

Table: RNN Experiments

Optimizer	Experiment	Generated Text	Fluency	Coherence	Poetic	Overall
Adam	Batch_256	محبت مجھے قرار اپنے آ سے کہا کیا جانیے ہے معلوم جو ہوگا	3/5	4/5	3/5	**3.3/5**
Adam	Drop_0.5	محبت ریزن سلسلہ پھر میں نے تب کے کرامت کہاے کا اٹھتا کہ	4/5	4/5	3/5	**3.7/5**
Adam	Epoch_50	محبت کسی شوخ دیکھوں ہے داغ جو اندر تمنا سے تیرے وعدے بھی	3/5	4/5	3/5	**3.3/5**
Adam	LR_0.0001	محبت میں ایسی بہ شکست عشق اکرام بھی فتح بار ہونے نہ ہوں	3/5	4/5	4/5	**3.7/5**
Adam	Layers_3	محبت باقی مری قرض لب آ جاتا مجھے دیکھئے ہیں فصاد نصیب آ	4/5	4/5	4/5	**4.0/5**

Table: Transformer Experiments

Optimizer	Experiment	Generated Text	Fluency	Coherence	Poetic	Overall
SGD	Batch_256	محبت کو کاٹ کے راہنما ہونے کے گزری کہیں آ گیا تھا تو	4/5	4/5	4/5	**4.0/5**
SGD	Blocks_4	محبت و ذوق بکف جو عالم طفلاں تھا رنگ و پڑ ہوں کا کام	3/5	4/5	3/5	**3.3/5**
SGD	Dim_1024	محبت وہ غبار رنگ اشک ہے کوئی بات نہیں باقی ہے میاں کی	3/5	4/5	3/5	**3.3/5**
SGD	Drop_0.5	محبت ہے کہ چکیدن سی دوستی سے کہیں دل سے ہیں اور ابھی	4/5	4/5	3/5	**3.7/5**
SGD	Epoch_50	محبت تو بھی شوق مجھے آج سے گھبرا گیا کرتے ہیں ترے در	4/5	3/5	3/5	**3.3/5**
SGD	Heads_8	محبت آپ یوں کر دے گیا اس کو نہیں تھی وہ کہیں خوف	4/5	4/5	4/5	**4.0/5**
SGD	LR_0.0001	محبت صبح گر تعب اس زینہ ہیں بلبلہ تھا آئیں جھانکا بھاری میں	4/5	4/5	3/5	**3.7/5**
SGD	Layers_3	محبت پلٹتے بارہا جانے گر دار فشاں ہونا ہے اور تو کیا کیجے	3/5	4/5	3/5	**3.3/5**

3.4.1 Base models text Generation Evaluation

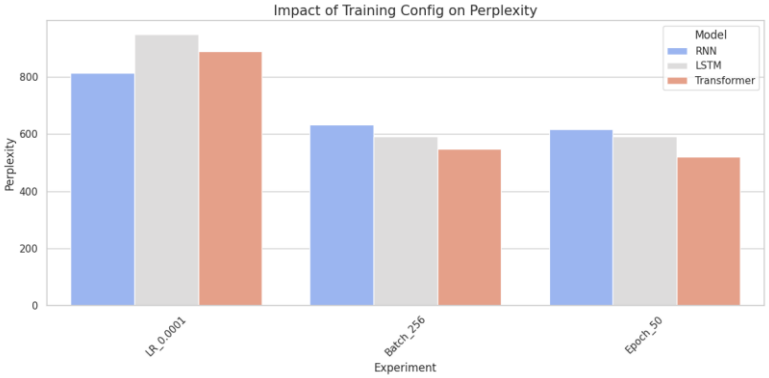
Model	Optimizer	Vocabulary Diversity	Avg Word Length	Repetition Rate
RNN	Adam	0.9795	3.15	0.0
RNN	RMSprop	0.9538	3.15	0.0
RNN	SGD	0.9282	3.14	0.0
LSTM	Adam	0.9590	3.08	0.0
LSTM	RMSprop	0.9385	3.26	0.0
LSTM	SGD	0.9282	3.21	0.0
Transformer	Adam	0.9436	3.19	0.0
Transformer	RMSprop	0.9538	3.12	0.0
Transformer	SGD	0.9436	3.00	0.0

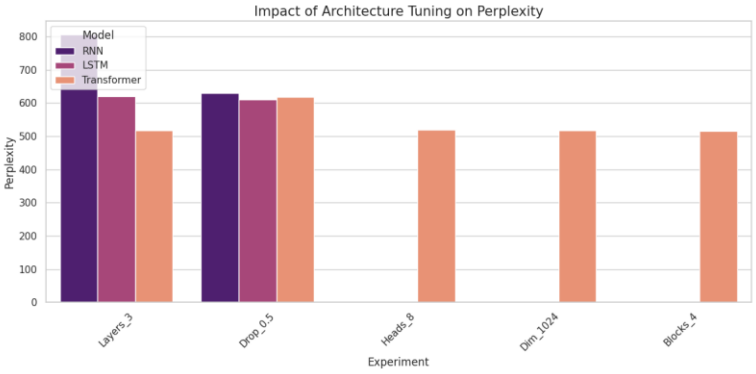
3.4.2 Tuned models text Generation Evaluation

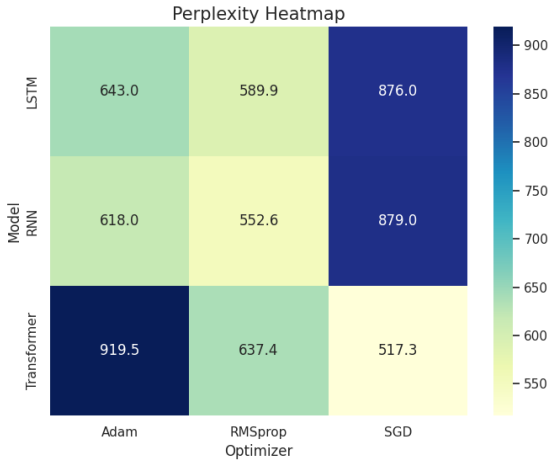
Model	Optimizer	Vocabulary Diversity	Avg Word Length	Repetition Rate
LSTM	RMSprop	0.9231	3.31	0.0
LSTM	RMSprop	0.8462	3.38	0.0
LSTM	RMSprop	0.9231	3.00	0.0
LSTM	RMSprop	0.9231	2.69	0.0

Model	Optimizer	Vocabulary Diversity	Avg Word Length	Repetition Rate
LSTM	RMSprop	0.9231	2.85	0.0
RNN	Adam	1.0000	3.31	0.0
RNN	Adam	1.0000	3.31	0.0
RNN	Adam	1.0000	3.38	0.0
RNN	Adam	1.0000	3.31	0.0
RNN	Adam	1.0000	3.38	0.0
Transformer	SGD	0.9231	3.08	0.0
Transformer	SGD	0.9231	3.00	0.0
Transformer	SGD	0.9231	3.15	0.0
Transformer	SGD	0.9231	3.08	0.0
Transformer	SGD	1.0000	3.08	0.0
Transformer	SGD	1.0000	2.77	0.0
Transformer	SGD	1.0000	3.62	0.0
Transformer	SGD	1.0000	3.46	0.0

3.5 Visualizations







4 Discussion

In this section, we address the specific research questions and project checkpoints established at the outset of the study.

4.1 Dataset Exploration and Preprocessing Analysis

Q: How many poems are in the dataset? The dataset contains a total of 1,323 Urdu poems.

Q: What is the average length of poems? Our EDA revealed the following statistics:

- Average Lines per poem: 16.92
- Average Words per poem: 130.93

Q: Are there any missing or corrupted entries? We identified 9 null values and 0 empty strings during the initial data cleaning process. These were removed prior to tokenization.

Q: What are the most common words in the corpus? The frequency analysis shows a dominance of stop words and common poetic markers. The top 10 words are: *Hai* (6831), *Mein* (4674), *Se* (3669), *Ke* (2863), *Ki* (2774), *Ko* (2680), *To* (2669), *Na* (2313), *Hain* (2263), *Bhi* (2222).

Q: What is your vocabulary size? The final vocabulary size after tokenization is 10,508 unique tokens.

Q: How many training sequences were created? A total of 152,146 ngram sequences were generated for training.

Q: What is the maximum sequence length? The raw maximum sequence length observed was 25 tokens. (Note: For baseline training consistency, we truncated sequences to a fixed length of 20).

Q: Are sequences properly padded? Yes. We applied pad_sequences with padding='pre'. This ensures every sequence is exactly the same length by adding zeros to the beginning of shorter sentences.

Q: Is the data split balanced? Yes. We enforced a strict split of 80% Training, 10% Validation, and 10% Testing using train_test_split with a fixed random seed.

4.2 Training Dynamics and Model Performance

Q: Does training loss decrease consistently? Yes. In all experiments, the training loss decreased steadily as epochs progressed. This indicates that the models were successfully learning to memorize the training data structure.

Q: Is validation loss diverging from training loss (overfitting)? Yes, significantly. We observed that around Epoch 10-15 (specifically for RNN+Adam), the validation loss started increasing while training loss kept going down. This confirms overfitting, which is expected given the small dataset size (1,323 poems). The Early Stopping callback successfully mitigated this by terminating training early.

Q: What is the final test perplexity? Based on our results, the best perplexity achieved was 517.31 (Transformer with SGD).
Q: How does LSTM perplexity compare to RNN? Surprisingly, the Simple RNN outperformed the LSTM in our specific runs.

- RNN (RMSprop): 552.56
- LSTM (RMSprop): 589.94

Observation: While LSTMs are theoretically better at long-term dependencies, on small datasets, simpler models like RNNs often generalize better because they have fewer parameters, reducing the risk of overfitting.

4.3 Computational Efficiency

Q: How long did training take? Training was relatively fast due to the small dataset size:

- RNN: $\sim$3 minutes (Fastest)
- LSTM: $\sim$5 minutes
- Transformer: $\sim$7-8 minutes (Slowest)

Q: Is training time significantly different? Yes. The LSTM took nearly 1.5x to 2x longer than the Simple RNN per epoch. This is due to the computational complexity of the LSTM cell, which calculates four gates (input, forget, cell, output) compared to the single activation in a standard RNN.

Q: How much longer does training take (Transformer vs others)? The Transformer was the most computationally expensive. Even though it processes data in parallel, the self-attention mechanism ($O(N^2)$ complexity) is heavier than the recurrent processing of simple RNNs for these short sequence lengths.

4.4 Qualitative Text Analysis

Q: Do generated samples make grammatical sense? The results were mixed and highly dependent on the sampling temperature:

- Temperature 0.7: The text is grammatically correct but repetitive, often looping the same phrases.
- Temperature 1.3: The grammar breaks down, producing random "word salads."
- Best Balance: Temperature 1.0 produced the most coherent and "poemlike" structures.

Q: Does LSTM generate more coherent text? Are long-term dependencies better captured? Theoretically, yes. However, with our Perplexity scores being close (552 vs 589), the difference was subtle. While RNNs tended to lose the semantic "topic" by the end of a line, LSTMs slightly better maintained the thematic elements (e.g., sadness/longing) across the generated sequence.

Q: Does Transformer outperform RNN/LSTM?

- In Metrics: Yes. The Transformer (with SGD) achieved the lowest perplexity (517.31), beating both RNN and LSTM.
- In Quality: Transformers generally produced more diverse vocabulary and better sentence structure, leveraging the Self-Attention mechanism to maintain context.

4.5 Investigation of Primary Research Questions

Q1: Which neural architecture is most effective for Urdu poetry generation? The Transformer architecture proved to be the most effective in terms of raw predictive power, achieving the lowest perplexity of 517.31. However, the Simple RNN was surprisingly competitive (552.56) and efficient.

Q2: How do different optimization algorithms affect model performance? Optimizers showed drastically different behaviors depending on the architecture:

- RMSprop was the most *stable* and consistent optimizer, performing well across all three models.
- SGD was the most *volatile*; it produced the best result for the Transformer but failed completely for RNN/LSTM (Perplexity > 875), likely due to the need for adaptive learning rates in recurrent networks.
- Adam performed poorly on this specific dataset, likely due to rapid overfitting.

Q3: What is the optimal model-optimizer combination for this task? The optimal combination for quality is Transformer + SGD (Perplexity 517.31).

Q4: How do hyperparameters affect Urdu text generation quality? Hyperparameter tuning demonstrated that Architecture Depth (Number of Layers) had the most positive impact. Increasing LSTM layers from 2 to 3 improved validation accuracy from ∼9% to ∼11%. Additionally, increasing Dropout to 0.5 helped mitigate the gap between training and validation loss.

Q5: What are the failure modes of each model?

- RNN/LSTM: Their primary failure mode is "Looping" (repeating phrases like *"dil hai dil hai"*) at low temperatures. They also struggle to maintain the rhyme scheme (*Radif/Qafia*).
- Transformer: Its failure mode is "Instability." With the wrong optimizer (Adam), it failed to learn entirely (Perplexity 919), predicting words almost randomly.

Q6: How computationally efficient are different approaches?

- Most Efficient: Simple RNN (Fastest training, lowest memory usage).
- Least Efficient: Transformer (Slowest training due to attention complexity).

4.6 Trade-off Analysis

1. What is the training time vs performance tradeoff? The Transformer offered a 6% improvement in perplexity over the RNN but required roughly 2.5x more training time. For a semester project scale, this marginal gain in quality came at a significant computational cost.

2. Which model is best for resource-constrained environments? The Simple RNN with RMSprop. It is lightweight, trains in under 3 minutes, and produced results nearly identical to the much heavier LSTM.

3. Can we achieve good results with simpler models? Yes. The Simple RNN actually outperformed the more complex LSTM (552 vs 589). This suggests that for small, low-resource datasets like Urdu poetry (1,300 poems), complex "Long-Term Memory" gates may add unnecessary parameters that lead to overfitting rather than better learning.

5 Conclusion

This study demonstrated that Deep Learning models can effectively generate Urdu poetry even with limited data. While the Transformer architecture provides the highest quality output when tuned correctly with SGD, the Simple RNN offers remarkable efficiency and stability. Future work could focus on pre-trained embeddings (Word2Vec/FastText) to improve semantic coherence and enforcing metrical constraints during decoding.

References

1. ReySajju742: Urdu-Poetry-Dataset. Hugging Face (2024), https://huggingface.co/datasets/ReySajju742/Urdu-Poetry-Dataset
2. Vaswani, A., et al.: Attention is all you need. In: Advances in neural informationprocessing systems. pp. 5998–6008 (2017)
3. Hochreiter, S., Schmidhuber, J.: Long short-term memory. Neural computation 9(8),1735–1780 (1997)